



OUTLIERS

It Still Needs You

Part 4 of 4 · Getting it to run on its own,
and to tell you when it stops.

Why do you still have to remember to use it?

Everything in it works, and all of it works when you sit down and start it. Which means you have not built a system yet. You have built a tool, and you are still the one operating it.

YOU WILL RECOGNISE SOME OF THIS

- You forget it exists for a week at a time.
- You set something up to run by itself and have no idea whether it still does.
- You cannot tell whether it did nothing because there was nothing to do, or because it broke a fortnight ago.
- Something once opened a window in the middle of your afternoon, and you turned it off and never turned it back on.
- You start each morning deciding what to look at, which is the job it was supposed to be doing for you.

WHY IT HAPPENS

Nothing here runs unless you start it, and nothing tells you when it stops.

The second half is worse than it sounds. **A job that ran and produced nothing looks exactly like a job that had nothing to do.** Both are silent. So a thing can fail on a Tuesday and carry on reporting success for a month, and there is nobody sitting beside you to notice. Mine did precisely that: it ran sixty-four times, produced nothing every time, and said it had worked.

And the window that opened in the middle of your afternoon is not a small thing. A job that interrupts you gets switched off within a week, and a job that is off is a job that is not running.

What fixes it

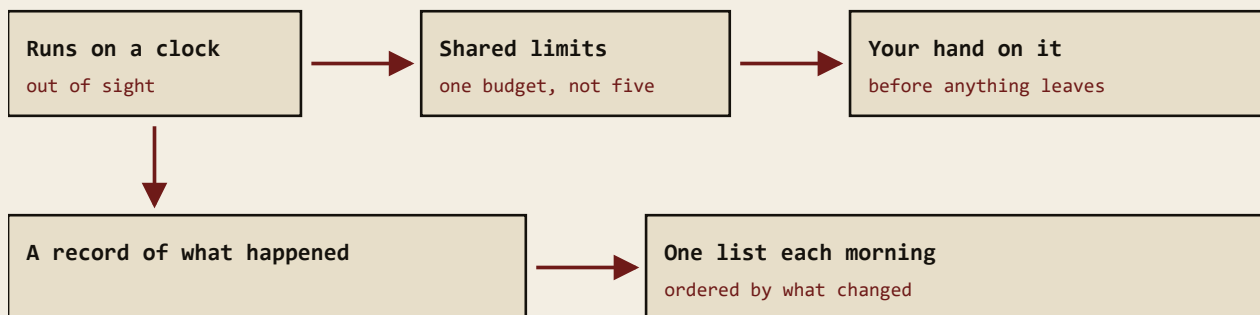
A timetable, a gate, a record of what happened, and one short list each morning.

A timetable. Things run at a set time rather than when you remember, and they run out of sight - no window, nothing taking your keyboard mid-sentence.

A gate. Anything meant for another person is written, checked by something other than the thing that wrote it, and then stops. You send it.

A record. A running list of what happened and when, added to and never rewritten. Your notes describe how things are; the record describes what occurred.

One list each morning, ordered by what has changed, each line saying why it is there.



Nothing appears on your screen while you work. That is the difference between automation you keep and automation you

A CLOCK, A GATE, A RECORD, AND ONE LIST

WHY THIS WAY AND NOT ANOTHER

- **It runs invisibly, and that is not cosmetic.** The most common reason automation stops working is that somebody turned it off because it was annoying them.
- **The gate is built into the machinery, not written into instructions.** Instructions are the first thing a busy system stops reading. The work stops at a queue and a person moves it.
- **Nothing marks its own homework.** Whatever wrote a thing does not get to approve it. A second pass by something else is the cheapest check there is and the one most often skipped, because the first pass sounded confident.
- **Silence is not evidence that things are fine.** So the last thing installed is not another feature - it is a comparison between what should be true and what is, which says so loudly when they differ.

Do this

One command. It asks you a few questions so what you get is yours, then does the work.

```
git clone https://github.com/OUTLIERS-ai/outliers-sb-04-operations
cd outliers-sb-04-operations
python install.py
```

This needs Parts 1 to 3 already done. It arranges things that already work, so there has to be something to arrange.

WHAT IT ASKS YOU, AND WHY

QUESTION	WHAT IT CHANGES
When do you want the morning list?	Sets the time it is ready. It is prepared quietly beforehand, not while you read it.
Your hand on anything that reaches a person?	Yes is the default and the recommendation. No is possible, and is a decision worth making deliberately rather than by accepting a default.
Where should it tell you when something breaks?	Somewhere you already look. A warning somewhere you do not look is not a warning.

WHAT YOU END UP WITH

A timetable, a gate before anything leaves, a record that is added to and never rewritten, one list each morning, and a check that compares what should be true against what is.

What you will notice

Read one list in the morning. Look at the warnings when they come.

The list is short on purpose and each line says why it is there. It is ordered by what has changed rather than by what you added first, because something that arrived this morning matters more than something that has been sitting in a queue for a fortnight.

The warnings are rarer and worth more. Each names something that should have been true, what was actually found, and where to look. **A warning you cannot act on is a fault in the warning**, not something to get used to.

IF YOU ALREADY HAVE SOMETHING LIKE THIS

If you already have things running on a schedule, here is the test: name which of them are supposed to be running right now. Most people cannot, because a job switched off deliberately and a job that broke look identical from outside. This writes down which are meant to be on, and then checks.

WHAT IT STILL DOES NOT DO

Nothing further to install. What changes is that it starts telling you things, and what it tells you sends you back to the earlier parts: a rule that keeps misfiring gets rewritten, a source that never produces anything useful gets dropped.

The other thing that changes is what you can now build on top. Something that remembers, keeps itself honest, fills itself and runs without watching is solid enough for the next thing to stand on - and the first one worth standing there is the one that tracks your dealings with other people.

NEXT: PART 1 • MEMORY

Back to the beginning with what you have learned. This is the last of the four, and not the end of the build: it is the point where the thing starts improving from what it reports rather than from the next instalment.

WORDS USED HERE

WORD	WHAT IT MEANS
Scheduled task	A job your computer runs at a set time without being asked. Run out of sight, or it interrupts you and gets switched off.
Ledger	A record only ever added to, never rewritten, so what it said last month still says the same thing today.